Lista de Compras

Explicação didática do código

e perguntas prováveis do professor, com respostas

Paradigma de Programação — Trabalho Bimestral

Aguinaldo Alves · Gustavo Pereira · Gustavo Miguel · João Pedro

Novembro de 2026

Como usar este material

Este documento explica o código do sistema **na ordem em que ele funciona**: primeiro o que acontece quando o programa abre, depois as classes que guardam os dados (Model), as telas (View), quem liga as telas aos dados (Controller) e as classes de apoio. No fim há perguntas que o professor pode fazer na apresentação, cada uma com a resposta e com o nome de quem do grupo deve responder.

> **Dica de estudo:** leia uma seção com o código aberto no IntelliJ ao lado. Use `Ctrl + clique` no nome de um método para ir até ele, e `Alt + F7` para ver onde ele é usado.

1. Visão geral: MVC e pacotes
2. O que acontece quando o programa abre
3. Model: as classes de dados e as regras
4. View: FXML, Scene Builder e CSS
5. Controller: a ponte entre a tela e os dados
6. Classes de apoio (pacote `util`)
7. Três cliques acompanhados do começo ao fim
8. Recursos da linguagem Java usados no projeto
9. Testes
10. Limitações conhecidas (para responder com honestidade)
11. Perguntas prováveis do professor e respostas
12. Glossário rápido

1. Visão geral: MVC e pacotes

O sistema está dividido no padrão **MVC** (Model–View–Controller). A ideia é separar três responsabilidades, para que mexer em uma não obrigue a mexer nas outras:

Parte	Responsabilidade	Onde está
Model	Os dados e as **regras de negócio** (nome obrigatório, nome repetido, cálculo do total, gravação no arquivo). Não sabe nada de tela.	`br.com.listacompras.model`
View	O visual: botões, campos, tabelas. Não tem regra nenhuma.	`src/main/resources/view/*.fxml` e `css/estilo.css`
Controller	Recebe os cliques da tela, pede ao Model para fazer a operação e atualiza a tela com o resultado.	`br.com.listacompras.controller`
Apoio	Código reaproveitado por várias telas (trocar de tela, janelas de mensagem, formatar R$).	`br.com.listacompras.util`

```
Lista_Compras/
├── pom.xml                          ← dependências: JavaFX 21 e Gson
├── dados/lista_compras.json         ← criado na primeira execução
└── src/main/
    ├── java/
    │   ├── org/example/Main.java    ← ponto de partida
    │   └── br/com/listacompras/
    │       ├── App.java             ← inicia o JavaFX
    │       ├── model/               ← Cadastro, ListaCompra, Estabelecimento, Produto,
    │       │                          ItemLista, UnidadeMedida, Repositorio, Persistencia,
    │       │                          ValidacaoException
    │       ├── controller/          ← um controller por tela + ExclusaoCadastro
    │       └── util/                ← Navegador, Mensagem, BotoesLinha, Formatador, Encerramento
    └── resources/
        ├── view/                    ← 5 telas .fxml (Scene Builder)
        ├── css/estilo.css
        └── img/produto_padrao.png   ← imagem de produto sem foto
```

2. O que acontece quando o programa abre

1. `Main.main()` é executado. Ele só chama `App.main()`.
2. `App.main()` chama `launch(args)`, que liga o JavaFX. O JavaFX cria a janela (o `Stage`) e chama `App.start(palco)`.
3. Em `start`, antes de mostrar qualquer tela, o sistema carrega os dados do arquivo JSON: `Repositorio.getInstancia().ativarGravacao(new Persistencia(...))`.
4. O `FXMLLoader` lê o `MenuPrincipal.fxml`, monta os componentes e cria o `MenuPrincipalController`.
5. A tela é colocada numa `Scene`, a cena é colocada na janela e a janela aparece com `palco.show()`.

```
public void start(Stage palco) throws Exception {
    try {
        Repositorio.getInstancia().ativarGravacao(new Persistencia(Persistencia.ARQUIVO_PADRAO));
    } catch (ValidacaoException erro) {
        Mensagem.avisar(erro.getMessage());      // arquivo corrompido (RN077)
    }
    Parent raiz = FXMLLoader.load(getClass().getResource("/view/MenuPrincipal.fxml"));
    palco.setScene(new Scene(raiz));
    palco.setOnCloseRequest(evento -> {          // "X" da janela
        if (!Encerramento.confirmarSaida(palco.getScene())) {
            evento.consume();                    // consume() cancela o fechamento
        }
    });
    palco.show();
}
```

> **Por que existem duas classes de início (Main e App)?** Quando a classe principal herda de `Application` e o JavaFX está no classpath (como no Maven), o Java recusa abrir com o erro "JavaFX runtime components are missing". Começando por uma classe comum (`Main`) que chama o `App`, o problema some.

A metáfora do JavaFX é de teatro: **Stage** (palco) é a janela, **Scene** (cena) é o conteúdo que está no palco agora, e trocar de tela é trocar a cena.

3. Model: as classes de dados e as regras

3.1 `Cadastro`: a classe pai (herança e abstração)

Lista, estabelecimento e produto têm coisas em comum: um **id**, um **nome** e a regra de que o nome não pode se repetir. Em vez de escrever isso três vezes, criamos uma classe pai.

```
public abstract class Cadastro {
    private int id;                 // ENCAPSULAMENTO: ninguém de fora mexe direto
    private String nome;

    protected Cadastro(int id, String nome) {   // só as filhas chamam (super)
        this.id = id;
        this.nome = nome;
    }

    public boolean temMesmoNome(String outroNome) {
        if (nome == null || outroNome == null) return false;
        return nome.trim().equalsIgnoreCase(outroNome.trim());
    }

    public abstract String getDetalhe();            // cada filha responde do seu jeito
    public abstract String getMensagemEmUso();
    public abstract boolean estaVinculadoA(ItemLista item);

    @Override
    public String toString() { return nome; }       // o ComboBox mostra o nome
}
```

- **`abstract class`**: não dá para fazer `new Cadastro(...)`. Faz sentido, porque não existe "um cadastro qualquer"; todo objeto é uma lista, um estabelecimento ou um produto.
- **`abstract` nos métodos**: o pai diz *que* o método existe, mas não diz *como* funciona. Cada filha é **obrigada** a escrever a sua versão (senão o código não compila).
- **`temMesmoNome`**: `trim()` tira os espaços do começo e do fim, e `equalsIgnoreCase` compara ignorando maiúsculas e minúsculas. Assim " compras DA semana " é igual a "Compras da semana" (RN007).
- **`toString()`**: o `ComboBox` do JavaFX usa o `toString()` para escrever cada opção. Sem isso, apareceria algo como `ListaCompra@1b6d3586`.

3.2 As três filhas: `ListaCompra`, `Estabelecimento` e `Produto`

```
public class ListaCompra extends Cadastro {
    public ListaCompra(int id, String nome) {
        super(id, nome);           // chama o construtor do pai
    }

    @Override
    public String getDetalhe() {   // "3 produtos — R$ 92,00"
        Repositorio repositorio = Repositorio.getInstancia();
        List<ItemLista> itens = repositorio.itensDaLista(this);
        String produtos = (itens.size() == 1) ? "1 produto" : itens.size() + " produtos";
        return produtos + " — " + Formatador.moeda(repositorio.somarValores(itens));
    }

    @Override
    public String getMensagemEmUso() {
        return "Não é possível excluir: a lista possui itens vinculados";
    }

    @Override
    public boolean estaVinculadoA(ItemLista item) {
        return item.getIdLista() == getId();
    }
}
```

`extends` é a palavra da herança. `@Override` avisa ao compilador que o método está substituindo o do pai: se escrevermos o nome errado, dá erro de compilação em vez de um erro silencioso.

Repare que o detalhe da lista é **calculado na hora**, a partir dos itens. Por isso, quando um item é incluído na tela Criar Lista, a tela Cadastrar Lista já mostra o valor novo, sem precisar "avisar" nada.

`Estabelecimento`: os dias de promoção

```
private List<DayOfWeek> diasPromocao = new ArrayList<>();

private List<DayOfWeek> organizarDias(List<DayOfWeek> dias) {
    List<DayOfWeek> semRepetir = new ArrayList<>();
    if (dias != null) {
        for (DayOfWeek dia : dias) {
            if (dia != null && !semRepetir.contains(dia)) {   // RN023: sem repetir
                semRepetir.add(dia);
            }
        }
    }
    // No Java o domingo vale 7. "% 7" transforma o domingo em 0,
    // então ele passa a ser o primeiro da semana (RN026: Dom a Sáb)
    semRepetir.sort(Comparator.comparingInt(dia -> dia.getValue() % 7));
    return semRepetir;
}

public List<DayOfWeek> getDiasPromocao() {
    return new ArrayList<>(diasPromocao);   // devolve uma CÓPIA (encapsulamento)
}
```

- `DayOfWeek` é um enum do próprio Java (`MONDAY`, `TUESDAY`...). Não precisamos inventar uma forma de guardar dias.

- `getDiasPromocao()` devolve uma cópia: quem receber a lista pode mexer nela à vontade, que a lista original do objeto não muda.
- `getDetalhe()` transforma os dias em "Ter, Sex", ou devolve "Sem dias de promoção" (RN027).

`Produto`

O `nome` herdado guarda a **descrição**. Criamos `getDescricao()`, que só chama `getNome()`, para o código usar a mesma palavra que o usuário vê na tela. O produto também tem a `unidade` (enum `UnidadeMedida`) e o `caminhoFoto`, que pode ficar nulo (foto opcional).

3.3 `UnidadeMedida`: um enum

```
public enum UnidadeMedida {
    UNIDADE("Unidade"), QUILOGRAMA("Quilograma"), LITRO("Litro"),
    PACOTE("Pacote"), CAIXA("Caixa");

    private final String descricao;
    UnidadeMedida(String descricao) { this.descricao = descricao; }

    @Override
    public String toString() { return descricao; }   // ComboBox mostra "Pacote"
}
```

Um **enum** é um tipo com uma lista fechada de valores. O Java não deixa criar uma sexta unidade, e isso garante a RN038 ("a unidade deve ser escolhida entre unidade, quilograma, litro, pacote e caixa") sem nenhum `if`.

3.4 `ItemLista`: o produto dentro de uma lista

```
public class ItemLista {
    private int id;
    private int idLista;
    private Integer idEstabelecimento;   // Integer: pode ser null (RN051)
    private int idProduto;
    private UnidadeMedida unidade;       // unidade no momento da inclusão (RN056)
    private double quantidade;
    private Double valorUnitario;        // Double: pode ser null (RN054)

    public double getSubtotal() {
        if (valorUnitario == null) return 0;   // RN060
        return quantidade * valorUnitario;
    }
}
```

- **Guarda ids, não objetos.** Se o usuário renomeia o produto, todos os itens mostram o nome novo, porque buscam o produto pelo id na hora de mostrar. E o arquivo JSON não fica com cópias repetidas do produto.
- **`int` × `Integer`:** `int` é um tipo primitivo e não aceita `null`. `Integer` é a classe "embrulho" (wrapper) e aceita. Usamos `Integer` onde o campo é opcional.
- **Não herda de `Cadastro`:** o item não tem nome próprio e não é cadastrado sozinho; ele só existe dentro de uma lista. Herança só faz sentido quando a filha "é um" tipo do pai.

3.5 `ValidacaoException`: erros de regra de negócio

```
public class ValidacaoException extends RuntimeException {
    public ValidacaoException(String mensagem) {
        super(mensagem);
    }
}
```

Quando uma regra não é cumprida, o Model **lança** esta exceção com o texto exato do BDD. O controller **captura** e mostra o texto numa janela. Assim o Model não precisa saber que existe tela, e todas as mensagens ficam num lugar só.

3.6 `Repositorio`: onde os dados vivem e as regras são aplicadas

É a classe mais importante do Model. Ela guarda as **quatro listas** (os depósitos D1 a D4 do DFD) e tem um método para cada operação do sistema.

```
public class Repositorio {
    private static final Repositorio INSTANCIA = new Repositorio();   // o único objeto

    private final List<ListaCompra> listas = new ArrayList<>();              // D1
    private final List<Estabelecimento> estabelecimentos = new ArrayList<>(); // D2
    private final List<Produto> produtos = new ArrayList<>();                 // D3
    private final List<ItemLista> itens = new ArrayList<>();                  // D4

    private Repositorio() { }                       // ninguém de fora cria outro

    public static Repositorio getInstancia() { return INSTANCIA; }

    public List<ListaCompra> getListas() {
        return Collections.unmodifiableList(listas); // só leitura para quem está fora
    }
```

Singleton

O construtor é `private`, então só a própria classe consegue fazer `new Repositorio()`, e ela faz isso uma única vez, na constante `INSTANCIA`. Todas as telas pegam o mesmo objeto com `getInstancia()`. Esse padrão de projeto se chama **Singleton**, e é ele que faz as telas enxergarem os mesmos dados.

Cadastrar uma lista: as regras em ordem

```
public ListaCompra cadastrarLista(String nome) {
    String nomeLimpo = exigirTexto(nome, "Informe o nome da lista");                    // RN006
    recusarNomeRepetido(listas, nomeLimpo, null, "Já existe uma lista com esse nome");  // RN007

    ListaCompra lista = new ListaCompra(proximoId(listas), nomeLimpo);
    listas.add(lista);
    gravar();                                                                           // RN074
    return lista;
}

private void recusarNomeRepetido(List<? extends Cadastro> cadastros, String nome,
                                 Cadastro ignorar, String mensagem) {
    for (Cadastro cadastro : cadastros) {
        if (cadastro != ignorar && cadastro.temMesmoNome(nome)) {
            throw new ValidacaoException(mensagem);
        }
    }
}
```

- `List<? extends Cadastro>` quer dizer "uma lista de qualquer filha de Cadastro". Por isso o mesmo método serve para listas, estabelecimentos e produtos.
- O parâmetro `ignorar` serve para a edição: quem está sendo editado não conta como repetido de si mesmo (RN016: confirmar sem mudar o nome não acusa duplicidade).
- `proximoId` pega o maior id existente e soma 1.

Excluir: o exemplo principal de polimorfismo

```
public boolean estaEmUso(Cadastro cadastro) {
    for (ItemLista item : itens) {
        if (cadastro.estaVinculadoA(item)) return true;   // cada tipo compara o seu id
    }
    return false;
}

public void excluir(Cadastro cadastro) {
    if (estaEmUso(cadastro)) {
        throw new ValidacaoException(cadastro.getMensagemEmUso()); // cada tipo tem a sua frase
    }
    listas.remove(cadastro);
    estabelecimentos.remove(cadastro);
    produtos.remove(cadastro);
    gravar();
}
```

O método recebe um `Cadastro` e não sabe se é lista, estabelecimento ou produto. Em tempo de execução, o Java chama a versão de `estaVinculadoA` e de `getMensagemEmUso` da classe verdadeira do objeto. O `remove` só acha o objeto na lista do tipo dele; nas outras duas ele não faz nada.

Incluir item: soma em vez de repetir (RN055)

```
public ItemLista incluirItem(ListaCompra lista, Estabelecimento estabelecimento, Produto produto,
                             double quantidade, Double valorUnitario) {
    if (lista == null)   throw new ValidacaoException("Selecione a lista de compras");
    if (produto == null) throw new ValidacaoException("Selecione o produto");
    validarQuantidadeEValor(quantidade, valorUnitario);   // > 0 e valor >= 0

    for (ItemLista item : itensDaLista(lista, estabelecimento)) {
        if (produto.estaVinculadoA(item)) {      // o produto já está nesta lista e local
            item.somarQuantidade(quantidade);    // 6 + 2 = 8, sem criar outro item
            if (valorUnitario != null) item.setValorUnitario(valorUnitario);
            gravar();
            return item;
        }
    }
    ... cria um ItemLista novo, guardando produto.getUnidade() (RN056) ...
}
```

Os totais são somas simples: `somarQuantidades` soma as quantidades (RN061) e `somarValores` soma os subtotais (RN062).

Gravação a cada operação

Todo método que altera dados termina com `gravar()`, que pede à `Persistencia` para salvar o arquivo. Nos testes do Model a persistência fica nula, então nada é gravado.

3.7 `Persistencia`: o arquivo JSON com Gson

JSON é um formato de texto para guardar dados, com chaves e valores. **Gson** é uma biblioteca do Google, adicionada no `pom.xml`, que transforma objetos Java em JSON (`toJson`) e JSON em objetos (`fromJson`).

```
private static class Dados {           // o "formato" do arquivo: as quatro listas
    List<ListaCompra> listas = new ArrayList<>();
    List<Estabelecimento> estabelecimentos = new ArrayList<>();
    List<Produto> produtos = new ArrayList<>();
    List<ItemLista> itens = new ArrayList<>();
}

public void salvar(Repositorio repositorio) {
    Dados dados = new Dados();
    dados.listas = new ArrayList<>(repositorio.getListas());
    ...
    Files.createDirectories(arquivo.getParent());              // cria a pasta "dados"
    Files.writeString(arquivo, gson.toJson(dados), StandardCharsets.UTF_8);
}

public void carregar(Repositorio repositorio) {
    if (!Files.exists(arquivo)) {            // RN076: primeira vez, cria vazio
        ... salvar(repositorio); return;
    }
    try {
        Dados dados = gson.fromJson(Files.readString(arquivo), Dados.class);
        repositorio.substituirDados(...);
    } catch (IOException | JsonParseException | IllegalStateException erro) {
        guardarCopiaDoArquivoCorrompido();   // RN077: não perde o conteúdo
        throw new ValidacaoException("O arquivo de dados está corrompido ...");
    }
}
```

Para o Gson criar os objetos ao ler o arquivo, cada classe do Model ganhou um **construtor vazio privado**. O Gson usa *reflexão* (a capacidade do Java de olhar e preencher os campos de uma classe em tempo de execução) para chamar esse construtor e preencher os campos, mesmo sendo `private`.

```
{
  "listas": [ { "id": 1, "nome": "Compras da semana" } ],
  "estabelecimentos": [ { "diasPromocao": [ "TUESDAY", "FRIDAY" ], "id": 1, "nome": "Mercado
Central" } ],
  "produtos": [ { "unidade": "PACOTE", "id": 1, "nome": "Arroz branco 5kg" } ],
  "itens": [ { "id": 1, "idLista": 1, "idEstabelecimento": 1, "idProduto": 1,
               "unidade": "PACOTE", "quantidade": 2.0, "valorUnitario": 24.9 } ]
}
```

4. View: FXML, Scene Builder e CSS

As telas são arquivos **FXML**: um XML que descreve os componentes. Eles foram desenhados no **Scene Builder**, que gera esse XML, e por isso podem ser abertos e alterados visualmente.

```
<VBox fx:controller="br.com.listacompras.controller.CadastroListaController" ...>
    <TextField fx:id="campoNomeLista" promptText="Ex: Compras da semana" />
    <Button fx:id="btnAdicionar" text="+ Adicionar" onAction="#adicionarLista" />
    <TableView fx:id="tabelaListas"> ... </TableView>
</VBox>
```

No FXML	No controller	O que liga
`fx:controller="...CadastroListaController"`	a classe inteira	Qual classe controla esta tela
`fx:id="campoNomeLista"`	`@FXML private TextField campoNomeLista;`	O componente da tela vira um atributo do controller (o nome precisa ser igual)
`onAction="#adicionarLista"`	`@FXML private void adicionarLista(ActionEvent e)`	O clique chama este método

`@FXML` é uma **anotação**: avisa ao `FXMLLoader` que ele pode preencher aquele atributo (ou chamar aquele método) mesmo sendo `private`.

CSS

O `estilo.css` define as cores e formatos (classes como `.botao-principal` e `.cartao`). Um exemplo: os botões dos dias de promoção são `ToggleButton` (ficam pressionados), e a pseudo-classe `:selected` pinta de verde o dia marcado:

```
.botao-dia:selected {
    -fx-background-color: #2F7A48;
    -fx-text-fill: white;
}
```

5. Controller: a ponte entre a tela e os dados

5.1 O ciclo de vida de um controller

1. O `FXMLLoader` lê o FXML e cria o controller.
2. Preenche os atributos `@FXML` com os componentes da tela.
3. Chama `initialize()`, porque o controller implementa a interface `Initializable`. É ali que configuramos as tabelas e carregamos os dados.

> Não dá para usar o construtor do controller para isso, porque quando o construtor roda os campos `@FXML` ainda estão `null`. O `initialize()` roda **depois** que eles foram preenchidos.

5.2 `CadastroListaController`: o modelo das telas de cadastro

```
@Override
public void initialize(URL url, ResourceBundle recursos) {
    // Como cada coluna pega o valor de uma linha (lambda)
    colunaNomeLista.setCellValueFactory(linha -> new
SimpleStringProperty(linha.getValue().getNome()));
    colunaDetalhes.setCellValueFactory(linha -> new
SimpleStringProperty(linha.getValue().getDetalhe()));

    // Botões Montar, Editar e Excluir em cada linha (RN080)
    BotoesLinha.configurarColuna(colunaAcoes, this::montarLista, this::editarLista,
this::excluirLista);

    tabelaListas.setPlaceholder(new Label("Ainda não há listas cadastradas"));   // RN012
    atualizarTabela();
}

@FXML
private void adicionarLista(ActionEvent evento) {
    try {
        if (listaEmEdicao == null) {
            repositorio.cadastrarLista(campoNomeLista.getText());
            Mensagem.informar("Lista cadastrada com sucesso");
        } else {
            repositorio.alterarLista(listaEmEdicao, campoNomeLista.getText());
            Mensagem.informar("Lista alterada com sucesso");
        }
        limparFormulario();
        atualizarTabela();
    } catch (ValidacaoException erro) {
        Mensagem.avisar(erro.getMessage());    // texto do BDD vindo do Model
    }
}
```

- `setCellValueFactory` diz à coluna como obter o texto de cada linha. `linha -> ...` é uma **lambda**: uma função curta escrita no próprio lugar.
- `this::editarLista` é uma **referência de método**: passa o método como se fosse um valor, para ser chamado depois, quando o botão for clicado.
- **Modo de edição:** o atributo `listaEmEdicao` guarda qual lista está sendo editada. Se for `null`, o botão cadastra; se não, altera. Ao clicar em Editar, o nome vai para o campo, o botão vira "Salvar" e aparece "Cancelar".
- **O controller não tem regra de negócio:** ele não verifica se o nome está vazio nem se repete. Quem faz isso é o Repositorio. O controller só mostra o resultado.

As telas de **Estabelecimento** e **Produto** seguem o mesmo modelo. O Estabelecimento usa um `Map<DayOfWeek, ToggleButton>` para ligar cada botão de dia ao dia da semana do Java, e assim descobre quais dias estão marcados com um `for`. O Produto monta um **cartão** (`VBox` com foto, descrição, unidade e botões) para cada produto, usa `FileChooser` para escolher a foto no computador e `TextInputDialog` para informar um link de imagem.

5.3 `ExclusaoCadastro`: uma exclusão para as três telas

```
static boolean excluir(Cadastro cadastro, String tipo) {
    if (!Mensagem.confirmar("Deseja realmente excluir " + tipo + " \"" + cadastro.getNome() +
"\"?")) {
        return false;                          // RN018/RN033/RN047: pede confirmação
    }
    try {
        Repositorio.getInstancia().excluir(cadastro);
        return true;
    } catch (ValidacaoException erro) {
        Mensagem.avisar(erro.getMessage());   // a frase de cada tipo (polimorfismo)
        return false;
    }
}
```

5.4 `CriarListaController`: a Movimentação

- **Listas suspensas com objetos:** `ComboBox<ListaCompra>`, `ComboBox<Estabelecimento>` e `ComboBox<Produto>` guardam os próprios objetos, não textos. O que aparece escrito é o `toString()`.
- **"Sem estabelecimento":** como o estabelecimento é opcional (RN051), a primeira opção da lista é um objeto especial, a constante `SEM_ESTABELECIMENTO`, que nunca vai para o Repositorio. O método `estabelecimentoEscolhido()` troca essa opção por `null`.
- **Ouvintes (listeners):** ao trocar a lista ou o estabelecimento, a tabela é atualizada sozinha (RN058):

  ```
  comboLista.valueProperty().addListener((propriedade, antes, depois) -> atualizarTabela());
  ```

- **Ler números digitados:** `lerNumero` aceita "4,50", "4.50" e "1.234,50". Texto que não é número vira `null`, e o sistema mostra a mensagem do BDD.
- **Totais:** `atualizarTabela()` pega os itens filtrados e preenche "TOTAL DE ITENS" e "VALOR TOTAL" usando o Repositorio e o `Formatador`.
- **Receber a lista de outra tela:** o método público `setListaSelecionada(lista)`, explicado na seção 7.

5.5 `MenuPrincipalController`

Cada botão chama `Navegador.abrir(botao, "/view/....fxml")`. O botão Sair chama `Encerramento.confirmarSaida(...)` e, se a resposta for "Sim", `Platform.exit()` fecha o JavaFX.

6. Classes de apoio (pacote `util`)

6.1 `Navegador`: trocar de tela e devolver o controller

```
public static <T> T abrir(Node componenteDaTela, String caminhoFxml) {
    try {
        FXMLLoader carregador = new FXMLLoader(Navegador.class.getResource(caminhoFxml));
        Parent novaTela = carregador.load();
        Stage palco = (Stage) componenteDaTela.getScene().getWindow();   // a janela atual
        palco.setScene(new Scene(novaTela));                             // troca a cena
        return carregador.getController();                               // devolve o controller
    } catch (Exception erro) {
        erro.printStackTrace();
        Mensagem.avisar("Não foi possível abrir a tela " + caminhoFxml);
        return null;
    }
}
```

- `<T>` é um **método genérico**: o tipo de retorno é decidido por quem chama. Em `CriarListaController c = Navegador.abrir(...)`, o `T` vira `CriarListaController` sem precisar de conversão (cast).
- Usamos `new FXMLLoader(...)` em vez do atalho `FXMLLoader.load(...)` porque só o objeto carregador sabe devolver o controller com `getController()`.
- A partir de qualquer componente da tela, `getScene().getWindow()` chega à janela.

6.2 `Mensagem`: janelas de aviso e confirmação

```
public static boolean confirmar(String pergunta) {
    ButtonType sim = new ButtonType("Sim", ButtonData.YES);
    ButtonType nao = new ButtonType("Não", ButtonData.NO);
    Alert janela = new Alert(AlertType.CONFIRMATION, pergunta, sim, nao);
    Optional<ButtonType> resposta = janela.showAndWait();   // espera o clique
    return resposta.isPresent() && resposta.get() == sim;
}
```

`showAndWait()` para o código até o usuário responder. `Optional` é um "pode ter ou não ter valor": fica vazio se o usuário fechar a janela no X, por isso verificamos `isPresent()`.

6.3 `BotoesLinha`: botões dentro da linha da tabela (RN080)

```
coluna.setCellFactory(colunaDaTabela -> new TableCell<>() {
    private final HBox botoes = criar(
            () -> aoEditar.accept(getTableRow().getItem()),
            () -> aoExcluir.accept(getTableRow().getItem()));

    @Override
    protected void updateItem(Void item, boolean vazia) {
        super.updateItem(item, vazia);
        setGraphic(vazia ? null : botoes);   // linha vazia não mostra botões
    }
});
```

`setCellFactory` ensina a tabela a **desenhar** as células de uma coluna. Aqui criamos uma **classe anônima** (uma filha de `TableCell` escrita no próprio lugar, sem nome) que coloca os botões. Ao clicar, `getTableRow().getItem()` pega o objeto da linha e entrega para o método recebido (`Consumer<T>` é "uma função que recebe um T e não devolve nada").

6.4 `Formatador` e `Encerramento`

- `Formatador.moeda(92.0)` devolve "R$ 92,00" no padrão brasileiro; `quantidade(1.5)` devolve "1,5"; `contador(3, "lista cadastrada", "listas cadastradas")` devolve "3 listas cadastradas".
- `Encerramento.confirmarSaida(cena)`: procura campos de texto preenchidos na tela (cadastro em andamento, RN072), pergunta se o usuário quer sair (RN070) e grava o arquivo antes de fechar (RN071). É usado pelo botão Sair e pelo X da janela.

Essas classes têm **construtor privado** e só métodos `static`: são "caixas de ferramentas", ninguém precisa criar objetos delas.

7. Três cliques acompanhados do começo ao fim

7.1 Cadastrar a lista "Compras da semana"

1. O usuário digita o nome e clica em **+ Adicionar**. O FXML tem `onAction="#adicionarLista"`, então o JavaFX chama esse método no controller.
2. O controller chama `repositorio.cadastrarLista(campoNomeLista.getText())`.
3. O Repositorio verifica se o nome está vazio e se repete. Se houver problema, lança `ValidacaoException`; o controller captura e mostra a mensagem.
4. Se estiver tudo certo, cria o objeto `ListaCompra`, adiciona na lista D1 e chama `gravar()`. O JSON é atualizado.
5. O controller mostra "Lista cadastrada com sucesso", limpa o campo e chama `atualizarTabela()`, que recarrega a tabela e o contador.

7.2 Botão Montar: passagem de dados entre telas

1. Na tabela do Cadastro de Lista, o usuário clica em **Montar** na linha de "Compras da semana".
2. O `BotoesLinha` pega o objeto daquela linha e chama `montarLista(lista)`.
3. `montarLista` abre a tela nova e recebe o controller dela:

```
CriarListaController criarLista = Navegador.abrir(btnVoltar, "/view/CriarLista.fxml");
criarLista.setListaSelecionada(lista);   // entrega o MESMO objeto para a outra tela
```

4. `setListaSelecionada` faz `comboLista.setValue(lista)`. O listener da lista suspensa percebe a mudança e chama `atualizarTabela()`, mostrando os itens daquela lista.

7.3 Tentar excluir uma lista que já tem itens

1. Clique em **Excluir** na linha. O controller chama `ExclusaoCadastro.excluir(lista, "a lista")`.
2. Aparece "Deseja realmente excluir a lista "Compras da semana"?". O usuário clica em Sim.
3. `repositorio.excluir(cadastro)` chama `estaEmUso`, que percorre os itens e pergunta `lista.estaVinculadoA(item)`. A lista compara o `idLista` do item e encontra um.

4. O Repositorio lança `new ValidacaoException(cadastro.getMensagemEmUso())`. Como o objeto é uma `ListaCompra`, a mensagem é "Não é possível excluir: a lista possui itens vinculados".
5. O `ExclusaoCadastro` captura e mostra o aviso. A lista continua cadastrada.

8. Recursos da linguagem Java usados no projeto

Recurso	Exemplo no projeto	Para que serve
Herança (`extends`)	`ListaCompra extends Cadastro`	Reaproveitar atributos e métodos do pai
Classe e método abstratos	`abstract class Cadastro`, `abstract String getDetalhe()`	Obrigar as filhas a implementar
Sobrescrita (`@Override`)	`getDetalhe()` nas três filhas; `toString()`	A filha troca o comportamento do pai
Sobrecarga	`itensDaLista(lista)` e `itensDaLista(lista, estabelecimento)`; dois `configurarColuna`	Mesmo nome, parâmetros diferentes
Interface	Controllers `implements Initializable`	Contrato: "tenho o método initialize"
Enum	`UnidadeMedida`, `DayOfWeek`	Lista fechada de valores
Exceção própria	`ValidacaoException extends RuntimeException`	Avisar erro de regra com a mensagem pronta
Genéricos	`List<ListaCompra>`, `<T> T abrir(...)`, `List<? extends Cadastro>`	Código que funciona para vários tipos com segurança
Lambda	`linha -> new SimpleStringProperty(...)`	Função curta passada como valor
Referência de método	`this::editarLista`	Passar um método existente como valor
Classe anônima	`new TableCell<>() { ... }`	Criar uma filha sem nome no próprio lugar
Classe interna estática	`Persistencia.Dados`	Classe auxiliar usada só dentro de outra
`static` / Singleton	`Repositorio.getInstancia()`	Um único objeto compartilhado
Wrapper (`Integer`, `Double`)	`Integer idEstabelecimento`	Permitir `null` em campo opcional
`Optional`	Resposta do `Alert`	Valor que pode não existir
Pattern matching do `instanceof`	`componente instanceof TextField campo` (em `Encerramento`)	Testar o tipo e já criar a variável

9. Testes

Teste	Como funciona	Resultado
`TesteModel`	Usa só o Model, sem abrir tela: cria listas, produtos e itens e confere as regras e os valores (R$ 92,00 → R$ 103,00 → R$ 82,80)	52/52
`TesteTelasBDD`	Abre as telas de verdade e segue os cenários H01 a H18 do documento de análise, clicando nos botões. Um "relógio" (`Timeline`) olha a cada 50 ms se apareceu uma janela de mensagem, guarda o texto e clica na resposta combinada	91/91

Os dois rodam pelo IntelliJ: botão direito no arquivo > Run. O teste das telas grava numa pasta temporária, então não mexe no `dados/lista_compras.json`.

10. Limitações conhecidas

Se o professor perguntar "o que vocês melhorariam?", estas são respostas honestas:

- **Dinheiro em `double`:** `double` pode ter pequenos erros de arredondamento em contas com centavos. O correto em sistemas financeiros é `BigDecimal`. Usamos `double` pela simplicidade; a tela sempre arredonda para 2 casas.
- **Foto guardada pelo caminho:** o JSON guarda o caminho do arquivo da foto. Se a imagem for apagada ou movida, o sistema mostra a imagem padrão (RN090), mas a foto original se perde. Uma melhoria seria copiar a imagem para a pasta `dados`.
- **Singleton:** facilita a passagem de dados, mas deixa as classes dependentes de um objeto global, o que dificulta testes isolados.
- **Grava o arquivo inteiro a cada operação:** para o volume de uso pessoal é instantâneo (NF004), mas não escalaria para milhares de registros, onde um banco de dados seria o certo.
- **Aviso no console:** os FXML foram salvos por um Scene Builder mais novo (versão 26) e rodam no JavaFX 21. O aviso não afeta o funcionamento.

11. Perguntas prováveis do professor e respostas

A etiqueta à direita indica quem do grupo deve responder, pela divisão do trabalho. Todos devem conhecer as de POO.

11.1 Orientação a Objetos

1. Onde está a herança no sistema? Aguinaldo

`Cadastro` é a classe pai, e `ListaCompra`, `Estabelecimento` e `Produto` herdam dela com `extends`. As três herdam o `id`, o `nome` e o método `temMesmoNome()`. Assim a regra de nome repetido (RN007, RN022, RN037) foi escrita uma vez só. Também há herança em `ValidacaoException extends RuntimeException` e em `App extends Application`.

2. Por que `Cadastro` é abstrata? Aguinaldo

Porque não existe um "cadastro genérico": todo objeto é uma lista, um estabelecimento ou um produto. Sendo abstrata, ninguém consegue fazer `new Cadastro()`. Além disso, ela declara métodos abstratos (`getDetalhe`, `getMensagemEmUso`, `estaVinculadoA`) que cada filha é obrigada a implementar.

3. O que é polimorfismo e onde vocês usaram? Aguinaldo

Polimorfismo é chamar o mesmo método sobre uma variável do tipo do pai e cada objeto responder do seu jeito. No sistema: (1) a coluna "Detalhes" chama `getDetalhe()`, e a lista devolve "3 produtos — R$ 92,00", o estabelecimento "Ter, Sex" e o produto "Pacote"; (2) o método `excluir(Cadastro)` do Repositorio é um só para os três tipos e, se o cadastro estiver em uso, mostra `cadastro.getMensagemEmUso()`, que é a frase do BDD de cada tipo. O benefício é não ter `if` para descobrir o tipo e as três telas usarem o mesmo código.

4. Qual a diferença entre sobrescrita e sobrecarga? Tem as duas no projeto? Todos

Sobrescrita (override) é a filha reescrever um método do pai com a mesma assinatura: `getDetalhe()` nas três filhas e `toString()` em `Cadastro`. **Sobrecarga** (overload) é ter dois métodos com o mesmo nome e parâmetros diferentes na mesma classe: `itensDaLista(lista)` e `itensDaLista(lista, estabelecimento)` no Repositorio, e os dois `configurarColuna` do `BotoesLinha` (com e sem o botão Montar).

5. Como vocês aplicaram o encapsulamento? Aguinaldo

Todos os atributos são `private` e acessados por getters e setters. Além disso: `getDiasPromocao()` devolve uma cópia da lista, e o Repositorio devolve as listas com `Collections.unmodifiableList`. Com isso, ninguém de fora consegue alterar os dados sem passar pelas regras de validação.

6. Por que `ItemLista` não herda de `Cadastro`? João Pedro

Herança representa "é um". Um item não *é um* cadastro: ele não tem nome próprio, não é cadastrado sozinho e só existe dentro de uma lista. Ele é a associação entre lista, estabelecimento e produto (o depósito D4 do DFD).

7. Por que o item guarda os ids e não os objetos? João Pedro

Para não ter cópias. Se o produto for renomeado, todos os itens mostram o nome novo, porque buscam o produto pelo id na hora de exibir. No JSON também fica só o número, e não o produto inteiro repetido em cada item. É a mesma ideia de chave estrangeira em banco de dados.

8. Por que a unidade de medida é um enum? Gustavo Miguel

Porque a RN038 diz que a unidade só pode ser uma de cinco opções. O enum é um tipo com valores fixos, então o Java não deixa usar outro valor. Ele também guarda o texto bonito ("Pacote") que aparece no ComboBox, pelo `toString()`.

9. Qual a diferença entre classe abstrata e interface? Vocês usaram interface? Todos

A classe abstrata pode ter atributos e métodos já prontos (como `id`, `nome` e `temMesmoNome`), e uma classe só pode herdar de uma. A interface é um contrato de métodos, e uma classe pode implementar várias. Usamos as duas: os controllers implementam a interface `Initializable` do JavaFX, e para o Model escolhemos a classe abstrata porque as filhas compartilham atributos e código.

10. Para que serve o `super(id, nome)` e por que o construtor de `Cadastro` é `protected`? Aguinaldo

`super(...)` chama o construtor do pai, que guarda o id e o nome (os atributos são `private` no pai, então só ele pode atribuir). É `protected` porque só as classes filhas precisam chamá-lo.

11. Por que o `toString()` foi sobrescrito? Aguinaldo

O `ComboBox` usa o `toString()` do objeto para escrever cada opção. Sem sobrescrever, apareceria o nome da classe com um código (`ListaCompra@1b6d3586`). Sobrescrevendo, aparece o nome da lista.

11.2 JavaFX e Scene Builder

12. Como o FXML se liga ao controller? Gustavo Pereira

Por três ligações: `fx:controller` no elemento raiz diz qual classe controla a tela; `fx:id="campoNomeLista"` liga o componente ao atributo `@FXML private TextField campoNomeLista` (o nome tem que ser igual); e `onAction="#adicionarLista"` liga o clique ao método `adicionarLista`. Quem faz tudo isso é o `FXMLLoader`.

13. O que são Stage, Scene e Parent? — Gustavo Pereira

`Stage` é a janela. `Scene` é o conteúdo que está na janela. `Parent` é o componente raiz da tela (no nosso caso um `VBox`), de onde saem todos os outros. Para trocar de tela, o `Navegador` coloca uma `Scene` nova no mesmo `Stage`.

14. Por que configurar a tela no `initialize()` e não no construtor do controller? — Gustavo Pereira

Quando o construtor roda, o `FXMLLoader` ainda não preencheu os campos `@FXML`; eles estão `null`. O `initialize()` é chamado depois que todos os componentes já foram ligados.

15. Como a tabela sabe o que mostrar em cada coluna? — João Pedro

Pelo `setCellValueFactory`: para cada coluna passamos uma lambda que recebe a linha e devolve o valor. Por exemplo, a coluna Subtotal faz `Formatador.moeda(linha.getValue().getSubtotal())`. Os dados entram na tabela com `setItems(FXCollections.observableArrayList(...))`.

16. Como os botões Editar e Excluir ficam dentro da linha? — Gustavo Miguel

A classe `BotoesLinha` usa `setCellFactory`, que define como a célula é desenhada. Criamos uma `TableCell` que mostra um `HBox` com os botões. No clique, `getTableRow().getItem()` pega o objeto daquela linha e chama o método do controller (passado como `this::editarLista`). Nos produtos, os botões ficam dentro de cada cartão.

17. Por que existe a classe `Main`, se o `App` já tem um `main`? — Gustavo Pereira

Quando a classe principal herda de `Application` e o JavaFX está no classpath (como no Maven), o Java dá o erro "JavaFX runtime components are missing". Começando por uma classe que não herda de `Application`, esse problema não acontece.

11.3 MVC e passagem de dados

18. Explique o MVC no projeto de vocês. — Aguinaldo

Model é o pacote `model`: os dados e as regras. View são os FXML feitos no Scene Builder, sem regra nenhuma. Controller é o pacote `controller`: recebe o clique, chama o Model e atualiza a tela. Por exemplo, a regra "nome obrigatório" está no Repositorio, e não no controller: se amanhã a tela mudar, a regra continua valendo.

19. Como os dados passam de uma tela para outra? — Aguinaldo

De duas formas. (1) Todas as telas usam o mesmo objeto `Repositorio` (Singleton), então trabalham com os mesmos objetos: um item incluído no Criar Lista aparece na hora no total do Cadastro de Lista. (2) Passagem direta de objeto: o botão Montar faz `Navegador.abrir(...)`, que devolve o controller da tela nova, e chama `criarLista.setListaSelecionada(lista)`, entregando o objeto `ListaCompra` escolhido.

20. O que é o Singleton e por que usaram? Tem desvantagem? Aguinaldo

É um padrão em que a classe tem um único objeto para o sistema inteiro. No Repositorio, o construtor é `private` e o objeto fica numa constante `static`, entregue por `getInstancia()`. Usamos para que todas as telas compartilhem os mesmos dados. A desvantagem é que ele funciona como uma variável global, o que dificulta testar partes isoladas.

21. Se eu incluir um item no Criar Lista, como o Cadastro de Lista fica sabendo? Gustavo Pereira

Ele não precisa ser avisado. O `getDetalhe()` da lista calcula o texto na hora, a partir dos itens que estão no Repositorio. Quando a tela Cadastro de Lista é aberta, a tabela chama `getDetalhe()` e já mostra o valor novo.

22. Por que não existe uma tela de Consulta separada? Todos

O documento de requisitos do grupo (Processo de Software) diz que o menu tem exatamente cinco opções (RN002) e que a consulta é exibida "na própria tela de cadastro, sem exigir uma ação extra do usuário" (RN010). Então a Consulta exigida pelo trabalho de Java está nas tabelas e nos cartões de cada tela de cadastro, e na tabela de itens do Criar Lista.

11.4 Persistência

23. Onde e quando os dados são gravados? Aguinaldo

No arquivo `dados/lista_compras.json`, dentro da pasta do projeto. A gravação acontece a cada operação confirmada (RN074): todo método do Repositorio que altera dados termina com `gravar()`. Ao sair, o sistema grava de novo, por garantia (RN071). Ao abrir, o `App` carrega o arquivo antes de mostrar o menu (RN075).

24. Por que JSON? E o que é o Gson? Aguinaldo

JSON foi a decisão do documento de requisitos (NF001). É texto legível, dá para abrir e conferir. O Gson é uma biblioteca do Google que converte objetos Java em JSON (`toJson`) e JSON em objetos (`fromJson`), sem precisarmos escrever o arquivo na mão. Ela foi adicionada como dependência no `pom.xml`.

25. O que acontece se o arquivo não existir? E se estiver estragado? Aguinaldo

Se não existir, o sistema cria o arquivo vazio, sem mensagem de erro (RN076). Se estiver corrompido, o sistema avisa o usuário, abre sem dados e guarda uma cópia `lista_compras.json.corrompido`, para o conteúdo não se perder na próxima gravação (RN077).

26. Como o Gson consegue criar os objetos se os atributos são privados? Aguinaldo

Ele usa reflexão, um recurso do Java que permite inspecionar e preencher os campos de uma classe em tempo de execução. Cada classe do Model tem um construtor vazio privado só para isso: o Gson cria o objeto vazio e preenche os campos com os valores do arquivo.

11.5 Regras de negócio

27. Como o sistema impede nomes repetidos, mesmo com maiúsculas diferentes? — Gustavo Pereira

O método herdado `temMesmoNome()` compara com `trim()` (tira os espaços das pontas) e `equalsIgnoreCase()` (ignora maiúsculas e minúsculas). O Repositorio percorre os cadastros do mesmo tipo e, se achar um igual, lança `ValidacaoException("Já existe uma lista com esse nome")`. Na edição, o próprio registro é ignorado.

28. Onde ficam as validações: no controller ou no model? Por quê? — Todos

No Model (Repositorio). Assim a regra vale de qualquer lugar que chame o método, inclusive nos testes sem tela. O controller só captura a `ValidacaoException` e mostra a mensagem. A única validação no controller é transformar o texto digitado em número, porque isso é um assunto da tela.

29. Por que a exceção herda de `RuntimeException` e não de `Exception`? — Aguinaldo

`Exception` é "verificada": o Java obriga a declarar `throws` em todo método por onde ela passa. `RuntimeException` não obriga, o que deixa o código mais limpo. Mesmo assim, os controllers capturam com `try/catch` em todas as operações.

30. O que acontece se eu incluir um produto que já está na lista? — João Pedro

O sistema não cria um item repetido: soma a quantidade ao item existente (RN055). Se o leite tinha 6 e incluo 2, passa a ter 8. Isso vale para a mesma lista e o mesmo estabelecimento; em outro estabelecimento é outro item.

31. Como são calculados o subtotal e os totais? — João Pedro

Subtotal = quantidade × valor unitário; sem valor informado, o subtotal é zero (RN060). Total de itens é a soma das quantidades (RN061) e valor total é a soma dos subtotais (RN062), considerando só os itens exibidos (lista e estabelecimento escolhidos). Exemplo do BDD: 2 × 24,90 + 6 × 5,50 + 4 × 2,30 = R$ 92,00 e 12 itens.

32. Por que não dá para excluir um produto que está numa lista? Como o sistema sabe? — Gustavo Miguel

Para não deixar itens apontando para um produto que não existe mais (integridade dos dados). O método `estaEmUso` percorre os itens perguntando `produto.estaVinculadoA(item)`, que compara o `idProduto`. Se achar, lança a mensagem "Não é possível excluir: o produto está incluído em uma lista".

33. Se eu mudar a unidade de um produto que já está numa lista, o que acontece com os itens? — Gustavo Miguel

O sistema avisa que "o produto já está em listas montadas" (RN046) e os itens continuam como estavam, porque cada item guardou a unidade que o produto tinha no momento da inclusão (RN056). A quantidade lançada não muda.

34. Como funciona a foto do produto? — Gustavo Miguel

É opcional. O botão "Escolher foto" abre um `FileChooser` filtrado para JPEG e PNG; o botão "Usar link" aceita um endereço de imagem. O produto guarda o caminho ou o link. Se não houver foto ou a imagem não carregar, aparece a imagem padrão `produto_padrao.png` (RN040, RN090).

35. O que acontece quando clico em Sair? — Gustavo Pereira

A classe `Encerramento` pergunta se o usuário quer sair (RN070). Se houver algum campo digitado e não confirmado, a pergunta avisa disso (RN072). Se o usuário confirmar, o sistema grava o arquivo (RN071) e o `Platform.exit()` fecha a aplicação. O X da janela usa as mesmas regras.

11.6 Código e recursos do Java

36. O que significa `linha -> new SimpleStringProperty(...)`? E `this::editarLista`? — Todos

O primeiro é uma **lambda**: uma função curta, escrita no próprio lugar, que recebe `linha` e devolve o valor da coluna. O segundo é uma **referência de método**: passa o método `editarLista` como valor, para ser chamado depois, quando o botão for clicado.

37. O que é o `<T>` no `Navegador.abrir`? — Aguinaldo

É um método genérico: o tipo de retorno é decidido por quem chama. Escrevendo `CriarListaController c = Navegador.abrir(...)`, o Java entende que `T` é `CriarListaController` e não precisa de conversão (cast). Assim o mesmo método serve para abrir qualquer tela.

38. Por que `Double` (com D maiúsculo) no valor unitário? — João Pedro

Porque o valor é opcional. O tipo primitivo `double` não aceita `null`; a classe `Double` aceita. Com `null` sabemos que o usuário não informou o valor, o que é diferente de ter informado zero.

39. Como a opção "Sem estabelecimento" funciona, se ela não está cadastrada? — João Pedro

É um objeto `Estabelecimento` criado só para a tela (a constante `SEM_ESTABELECIMENTO`) e colocado como primeira opção do ComboBox. Ele nunca vai para o Repositorio: o método `estabelecimentoEscolhido()` troca essa opção por `null` antes de chamar o Model.

11.7 Testes, decisões e melhorias

40. Como vocês testaram o sistema? Todos

Com dois testes automáticos. O `TesteModel` testa as regras sem abrir telas (52 verificações). O `TesteTelasBDD` abre as telas e segue os cenários H01 a H18 do documento de análise, com os mesmos dados e mensagens, clicando nos botões (91 verificações). Todos passaram. Também testamos manualmente seguindo o roteiro da apresentação.

41. Se precisassem de um novo tipo de cadastro, por exemplo Categoria, o que mudaria? Aguinaldo

Criaríamos `Categoria extends Cadastro` e implementaríamos os três métodos abstratos. A regra de nome repetido, a exclusão com confirmação e o bloqueio de exclusão em uso já funcionariam sem mudar o `ExclusaoCadastro` nem o `excluir(Cadastro)`, graças à herança e ao polimorfismo. Faltaria a lista no Repositorio, a tela FXML e o controller.

42. O que vocês melhorariam? Aguinaldo

Usar `BigDecimal` para valores em dinheiro (o `double` pode ter erros de arredondamento), copiar a foto para a pasta `dados` em vez de guardar só o caminho, e, se o volume de dados crescer, trocar o arquivo JSON por um banco de dados. Ver a seção 10.

43. Abra o Scene Builder e mostre como um componente se liga ao código. Todos

Abrir `CriarLista.fxml`, selecionar o botão "+ Adicionar item" e mostrar, no painel *Code*, o `fx:id btnAdicionarItem` e o `On Action` `#adicionarItem`. Depois mostrar no `CriarListaController` o atributo `@FXML private Button btnAdicionarItem` e o método `adicionarItem`.

12. Glossário rápido

Termo	Significado
Atributo	Variável que pertence ao objeto (ex.: `nome`)
Classe abstrata	Classe que não pode virar objeto; serve de modelo para as filhas
Construtor	Método especial que roda ao criar o objeto com `new`
Controller	Classe que responde aos eventos de uma tela
Enum	Tipo com uma lista fixa de valores
Exceção	Objeto que representa um erro; é "lançado" com `throw` e "capturado" com `catch`
FXML	Arquivo XML que descreve uma tela JavaFX
Genérico	Código parametrizado por tipo, como `List<Produto>`
Gson	Biblioteca que converte objetos Java em JSON e vice-versa
JSON	Formato de texto para guardar dados em chaves e valores
Lambda	Função curta escrita no próprio lugar: `x -> x * 2`
Listener	Código que "escuta" uma mudança e reage a ela
Maven / pom.xml	Ferramenta que baixa as bibliotecas e compila o projeto
Polimorfismo	Mesma chamada, comportamentos diferentes conforme o objeto real
Reflexão	Capacidade do Java de examinar e alterar classes em tempo de execução
Scene / Stage	Conteúdo da janela / a janela
Singleton	Padrão em que a classe tem um único objeto
Wrapper	Classe que embrulha um primitivo: `Integer`, `Double`